

DLP Assignment 2

Muhammad Umar | 23K-0023

Colourization as Regression

Architecture of the Regression CNN

The regression network (RegressionCNN) takes a single-channel greyscale image as input and directly predicts continuous RGB values for each pixel. The model is built as an encoder-decoder, meaning it first compresses the image down and then expands it back up to full resolution. In total it has 6 convolutional layers, each using a 3×3 kernel. The table below breaks down every layer:

Layer	In Channels	Out Channels	Kernel	Other Ops
downconv1	1 (grey)	32	3×3	BatchNorm, ReLU, MaxPool2d(2)
downconv2	32	64	3×3	BatchNorm, ReLU, MaxPool2d(2)
rfconv	64	64	3×3	BatchNorm, ReLU
upconv1	64	32	3×3	BatchNorm, ReLU, Upsample($\times 2$)
upconv2	32	3	3×3	BatchNorm, ReLU, Upsample($\times 2$)
finalconv (MyConv2d)	3	3	3×3	—

The first two layers downsample the spatial resolution ($32\times 32 \rightarrow 8\times 8$) while increasing the number of feature channels, allowing the network to learn increasingly abstract representations. The middle layer (rfconv) processes these compact features without changing resolution. The final three layers then upsample back to 32×32 and reduce channels down to 3 RGB outputs. The last layer specifically uses the hand-implemented MyConv2d instead of PyTorch's built-in Conv2d.

How Many Epochs Was the Model Trained For?

The baseline regression model was trained for 25 epochs, with a batch size of 100, learning rate of 0.001, and the Adam optimizer. Training loss at the end of 25 epochs was 0.0087 and validation loss was 0.0088.

Effect of Training Duration

To understand how the number of training epochs affects performance, the model was trained under three different configurations: 5, 25, and 50 epochs. The loss values and output images from each run are compared below.

Epochs	Final Train Loss	Final Val Loss	Train/Val Gap
5	0.0131	0.0130	0.0001
25 (baseline)	0.0087	0.0088	0.0001
50	0.0069	0.0071	0.0002

Output images from each run (row 1 = grayscale input, row 2 = ground truth, row 3 = model prediction):

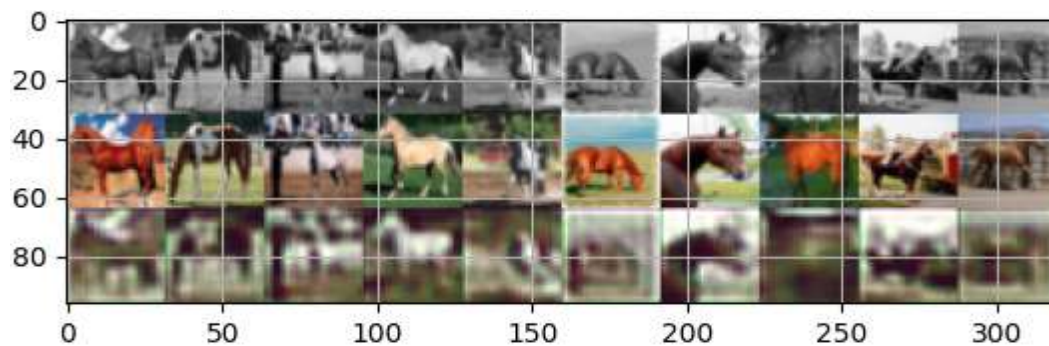


Figure 1 – Regression output after 5 epochs (Final Val Loss: 0.0130)

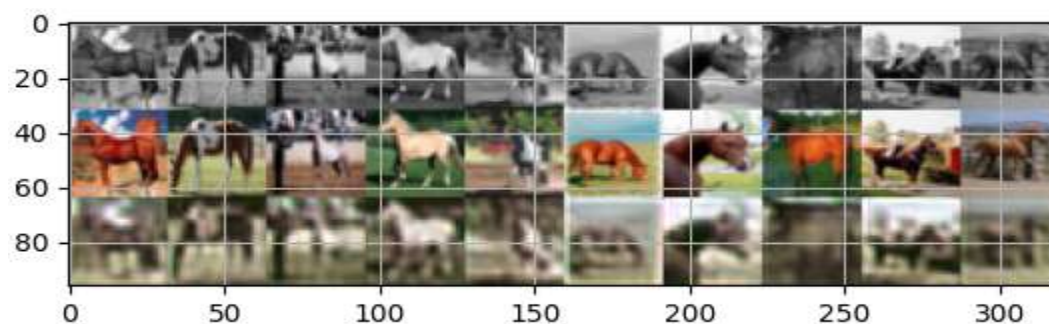


Figure 2 – Regression output after 25 epochs (Final Val Loss: 0.0088)

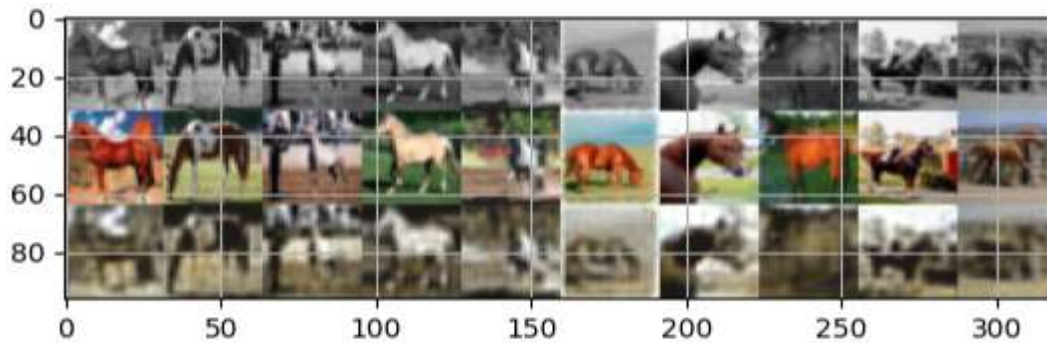


Figure 3 – Regression output after 50 epochs (Final Val Loss: 0.0071)

Looking at the numbers and the output images together, a few things stand out. The biggest drop in loss happens very early, just going from 5 to 25 epochs cuts the validation loss from 0.0130 to 0.0089. Going further to 50 epochs brings it down to 0.0071, which is a smaller gain relative to the extra compute. The training and validation losses stay very close throughout all three runs (gap never exceeds 0.0002), which tells us the model is not overfitting, it just has a natural performance ceiling.

That ceiling comes from the loss function itself. MSE pushes the model to predict the average colour across all possible valid colourings of each pixel. Since horses can be brown, black, grey, or white, the safest prediction in terms of squared error is a muted, desaturated mix of all of those, which is exactly what we see in the outputs, the silhouette is there, the shape is right, but the colours are washed out compared to the ground truth. More epochs refine this slightly but cannot fix the fundamental issue. That requires a different approach, which is addressed in the next section.

Colourization as Classification

Model Architecture

Instead of predicting continuous RGB values, the classification approach treats colourization as a per-pixel label prediction problem. Each pixel is assigned to one of 24 colour categories, determined by running k-means clustering on CIFAR-10 colour values (stored in `colour_kmeans24_cat7.npy`). The cross-entropy loss is then used to train the model.

The CNN model used here follows the exact same encoder-decoder structure as the regression model, with two important changes: (1) all Conv2d layers are replaced by the custom MyConv2d implementation, and (2) the final output has 24 channels instead of 3, one for each colour class. The layer breakdown is:

Layer	In Channels	Out Channels	Kernel	Other Ops
downconv1	1 (grey)	32	3×3	BatchNorm, ReLU, MaxPool2d(2)
downconv2	32	64	3×3	BatchNorm, ReLU, MaxPool2d(2)

rfconv	64	64	3×3	BatchNorm, ReLU
upconv1	64	32	3×3	BatchNorm, ReLU, Upsample(×2)
upconv2	32	24	3×3	BatchNorm, ReLU, Upsample(×2)
finalconv (MyConv2d)	24	24	3×3	—

Training Results for CNN (Classification)

The classification CNN was trained for 25 epochs under the same hyperparameter settings as the regression model. Results per epoch are summarised and the key final values are:

Metric	Epoch 1	Epoch 10	Epoch 25
Train Loss	2.4079	1.6596	1.5543
Val Loss	2.1518	1.6454	1.5690
Val Accuracy	26.1%	39.7%	41.5%

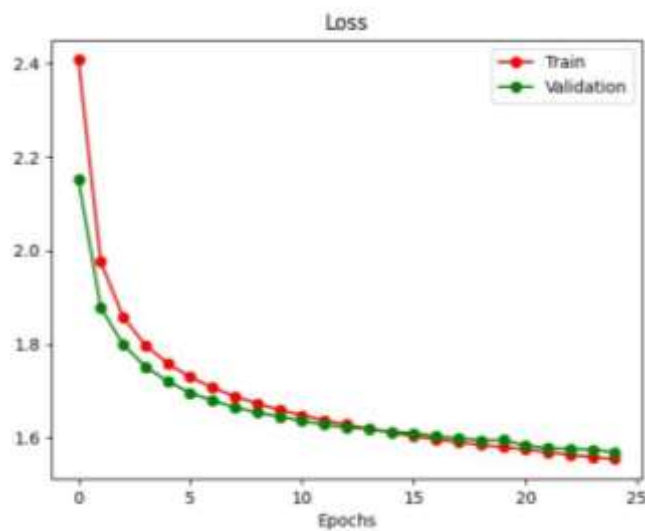


Figure 4 – Classification CNN training and validation loss curve over 25 epochs

Regression vs Classification — Which Performed Better?

The classification model is clearly a step up from the regression approach in terms of output quality. While a direct numerical comparison is not straightforward (MSE vs cross-entropy are different

scales), the qualitative difference in the generated images is noticeable. The classification model commits to specific discrete colours rather than predicting blended averages, so the output images have more visible colour contrast. Backgrounds tend to be more distinctly separated from the horse body, and the colours, while not always accurate, at least look like real colours rather than grey-brown smears.

The 41.5% pixel-wise validation accuracy also gives us a concrete measure that the regression model simply does not have. Given that random guessing over 24 classes would give ~4.2% accuracy, 41.5% represents meaningful learning. The model has genuinely learned to associate certain greyscale patterns with certain colour regions.

Skip Connections (the UNet)

What Are Skip Connections?

A skip connection takes the output of an encoder layer and feeds it directly into the corresponding decoder layer, bypassing the bottleneck in between. In the standard CNN, by the time the bottleneck representation is decoded back to full resolution, a lot of the fine spatial detail from the input has been lost. Skip connections fix this by giving the decoder a shortcut back to those earlier, high-resolution feature maps.

The UNet implemented here uses three skip connections. Specifically:

- The output of downconv2 (64 channels, 8×8) is concatenated with the input to upconv1, bringing its input from 64 to 128 channels.
- The output of downconv1 (32 channels, 16×16) is concatenated with the input to upconv2, bringing its input from 32 to 64 channels.
- The original greyscale input x (1 channel, 32×32) is concatenated with the input to finalconv, bringing its input from 24 to 25 channels.

Channel Dimensions with Skip Connections

Layer	Input Channels (with skip)	Output Channels	Spatial Size
downconv1	1	32	16×16
downconv2	32	64	8×8
rfconv	64	64	8×8
upconv1	$64 + 64 = 128$	32	16×16
upconv2	$32 + 32 = 64$	24	32×32
finalconv	$24 + 1 = 25$	24	32×32

Training Results for UNet

Metric	Epoch 1	Epoch 10	Epoch 25
Train Loss	2.3296	1.4624	1.2661
Val Loss	2.0618	1.4712	1.3375
Val Accuracy	30.3%	44.4%	49.0%

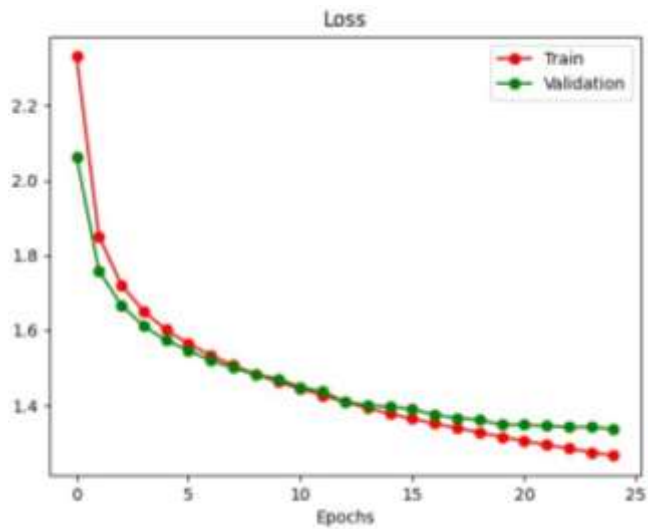


Figure 5 – UNet training and validation loss curve over 25 epochs

Did Skip Connections Help?

Yes, quite clearly. Comparing the final epoch numbers side by side:

Model	Final Val Loss	Final Val Accuracy
Classification CNN (no skip)	1.5690	41.5%
UNet (with skip connections)	1.3375	49.0%

The UNet achieves a 14.8% lower validation loss and 7.5 percentage points higher accuracy than the plain CNN, both trained for the same number of epochs with identical hyperparameters. The training curves also show that the UNet drops in loss faster in the early epochs by epoch 5, the UNet validation loss is already 1.5730, whereas the CNN is still at 1.7204 at the same point. This suggests the skip connections are not just helping at convergence but are actively making learning more efficient throughout training.

In terms of visual output, the UNet-generated colourizations show cleaner boundaries between the horse and the background, and colours tend to be more consistently applied to the right regions rather than bleeding across edges.

Two Reasons Why Skip Connections Improve CNN Performance

Reason 1 — Preserving Spatial Detail. When an image passes through pooling layers, the spatial resolution shrinks and fine details like edges and object boundaries are progressively compressed. By the time these features are being decoded back to full resolution, much of that precision is gone. Skip connections directly route early, high-resolution feature maps to the decoder, so the model does not have to reconstruct spatial detail from scratch. It can use these shortcuts to place colours accurately along boundaries rather than producing blurry colour blobs.

Reason 2 — Better Gradient Flow During Backpropagation. In a standard deep network, gradients have to pass through every layer during backpropagation, and they tend to shrink (vanish) as they travel further back. This makes it hard for the early layers to receive a strong enough learning signal. Skip connections create shorter paths for gradients to flow directly from the loss back to the early encoder layers. As a result, those layers get updated more effectively, and the overall network converges faster and to a better solution which is consistent with what we observed in the training curves.

Visualizing Intermediate Activations

1. CNN Activations (Early Layers vs Deep Layers)

Running the visualization block on test examples reveals a clear pattern in how the information is transformed as it moves through the network. The activations in the early layers look noticeably different from those deeper in the network.

In the first layer (out1, after downconv1), the activation maps are still fairly high-resolution and the features are visually interpretable. You can clearly see responses along the edges and outlines of the horse, each filter appears to be catching a different orientation of edges or contrast transitions. Some filters light up strongly along the top of the horse's back, others along the legs, and so on. The activations at this stage are essentially refined versions of the greyscale input.

By the second layer (out2, after downconv2), the resolution has halved again and the feature maps look much more abstract. Individual edges are no longer visible instead, the activations show broader, patch-like responses that roughly correspond to regions of the image (body, background, legs) rather than sharp boundaries. It is starting to encode what region a pixel belongs to rather than what local pattern it contains.

The bottleneck layer (out3, rfconv) operates at the smallest spatial size (8×8) and its activations are the most abstract. The maps look like blobs of activation distributed loosely across the image area. There is no obvious correspondence to any visual feature this layer is essentially encoding a compressed semantic summary of the entire image in a very small representation.

Moving into the decoder, the activations in out4 and out5 gradually become sharper again as resolution is restored. By out5 (32×32 again), the activation maps begin to resemble rough colour segmentation maps broad regions of activation corresponding to where specific colours will be predicted. The detail here is not sharp though, which is why regression and plain CNN outputs look a little blurry at the boundaries.

2. UNet Activations vs CNN Activations

The most noticeable difference between the CNN and UNet activations shows up in the decoder layers. Because the UNet receives skip-connected feature maps from the encoder, its decoder activations (out4, out5) are visibly sharper and more spatially structured than those of the plain CNN at the same stage.

In the CNN, the decoder has to reconstruct spatial structure from the compressed bottleneck alone. The result is that out4 and out5 activations tend to be spatially diffuse, the activation spreads loosely across regions without well-defined boundaries. In the UNet, the same decoder layers receive extra detail from the corresponding encoder layers via the skip connections, so the activations show much cleaner spatial organisation. Horse body boundaries are more clearly separated from the background, and specific body parts produce more localized activation patches.

The bottleneck activations (out3) are virtually identical between the two models, which makes sense since both architectures compress the input to the same 8×8 representation at that point and have not yet branched in how they decode it. The difference only becomes apparent once the decoding phase starts and the skip connections begin to contribute.

Overall, the UNet activations in the decoder tell a clearer story about the image structure, which directly translates into better-quality output colorizations.

5. Additional Theoretical Questions

Q1 — How Can Evaluation Be Improved Beyond Pixel-Level Metrics?

The loss functions used here (MSE for regression, cross-entropy for classification) both operate at the pixel level. They compare the model's prediction to the ground truth one pixel at a time, without any understanding of how a human would perceive the result. This creates a problem: a colorization that looks perfectly fine to a human might score poorly if the model chose a plausible but different color than the ground truth, while a washed-out average-color prediction might score well numerically because it is never too far from any possible ground truth.

A better evaluation strategy would incorporate perceptual similarity. Johnson et al. (2016) [2] proposed using a pretrained VGG network as a 'perceptual critic' instead of comparing raw pixel values, you pass both the predicted and ground-truth images through VGG and compare their intermediate feature representations. Features extracted by a network trained on ImageNet capture texture, style, and semantic content in a way that more closely mirrors human perception.

Other options include using SSIM (Structural Similarity Index), which factors in luminance, contrast, and structure together rather than averaging squared pixel differences, or FID (Fréchet Inception Distance), which measures how similar the distribution of generated images is to real images using deep features. At the end of the day though, none of these perfectly replace actually showing the outputs to people and asking them which looks more natural, that kind of human evaluation is expensive but remains the gold standard for image generation quality.

Q2 — How Can the Trained Model Be Applied to Images Larger Than 32×32 ?

Our models were trained entirely on 32×32 images, but this does not mean they are stuck at that resolution. Because the architecture is fully convolutional, every operation (conv, BatchNorm, ReLU, pooling, upsampling) works locally on spatial patches. The same learned weights can technically be applied to a larger input and will produce a proportionally larger output.

The main practical constraint is that the two MaxPool2d(2) layers mean the input width and height must both be divisible by 4. A 64×64 or 128×128 image would work fine, but odd-sized inputs would need to be padded first. The receptive field of each neuron stays the same regardless of input size, so the model would process a 128×128 image using the same local 3×3 filter patterns it learned on 32×32 images just applied across a larger canvas.

The BatchNorm statistics (mean and variance) were estimated from 32×32 training patches. At inference time these are frozen, so they still work on larger images, though there may be a slight mismatch in feature distribution for very large inputs. For best results on significantly larger images, one reasonable approach is to divide the image into overlapping 32×32 patches, colorize each independently, and then blend the results at the overlapping boundaries using weighted averaging to avoid visible seams. Alternatively, if a dataset of higher-resolution images were available, the model could be fine-tuned at the new resolution while keeping the pre-trained weights as a starting point.

References

- [1] Ronneberger, O., Fischer, P., and Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In International Conference on Medical Image Computing and Computer-Assisted Intervention (pp. 234–241). Springer, Cham.
- [2] Johnson, J., Alahi, A., and Fei-Fei, L. (2016). Perceptual losses for real-time style transfer and super-resolution. In European Conference on Computer Vision (pp. 694–711). Springer, Cham.